Midterm Project

Identifying Language Models from Scientific Ideas

The Pennsylvania State University

CMPSC 448

Jaiyoung Cho

Abstract

This project tests whether scientific research ideas reveal which language model generated them. A balanced subset of 9,000 responses from Claude Sonnet 4.6, GPT-5.4, and Gemini 3.1 Pro was used to train a CNN and a bidirectional LSTM. Keywords were separated across training, validation, and test sets. Response-only test accuracy reached 98.30% for CNN and 95.78% for BiLSTM; prompts alone gave 33.33%, and adding prompts did not consistently help. In transfer between life-science and physical-science topics, CNN accuracy remained 94.44-95.24%, while BiLSTM reached about 89.3%. Length-only, structural, and lexical classifiers achieved 73.26%, 89.70%, and 98.89%. Removing punctuation and keeping only the final 32 words left both neural models at 92.44%. These exploratory, single-seed results show that identifying information persists across the selected subjects and text transformations, while broader task transfer remains untested.

1. Problem and research questions

The task is three-class classification: given text, predict its generating model. Following the project questions, RQ1 asks whether the response alone supports attribution. RQ2 asks whether the prompt helps, using the same data splits and two architectures across three feature settings. RQ3 tests transfer between life-science and physical-science keywords. RQ4 examines response characteristics with interpretable classifiers and retrained text ablations.

2. Dataset construction

Scientific Ideation Diversity [1] is a public collection of model-generated research ideas. This project uses the three compressed idea files under ideas/effort-axis/. Only the standard structured_facet prompt and low reasoning-effort setting are retained. The output feature is the combined idea text. The input feature is reconstructed by substituting each recorded keyword into the published prompt template [2]. Raw API metadata, model-name fields, reasoning traces, and judge scores are excluded from classifier inputs.

Among 1,160 eligible shared keywords, 300 were sampled with seed 42. Ten responses per keyword per model produce 3,000 examples per model and 9,000 in total. The curation pipeline checks for empty text, invalid sample indices, failed structured-output validation, and duplicate samples. The saved metadata records no additional exclusions within the loaded low-effort subset.

Split	Keywords	Per model	Total
Training	210	2,100	6,300
Validation	45	450	1,350
Test	45	450	1,350

Table 1. Each keyword contributes ten examples from each model. All splits are balanced.

3. Preprocessing and model implementation

Split construction and text representation

Keywords are grouped before splitting, so responses to one keyword cannot appear in multiple splits. Keywords connected by a normalized exact duplicate response are kept in the same component. Duplicate matching uses Unicode normalization, case folding, and collapsed whitespace. The saved splits passed checks for unique record identifiers, correct label mappings, and no shared normalized keywords, prompts, or exact response texts. These checks do not detect every semantic paraphrase.

The tokenizer lowercases text and separates Unicode words and punctuation. A vocabulary is built from the training split only, with minimum frequency two and a cap of 30,000 entries including padding, unknown-token, and separator tokens. Each feature setting has its own vocabulary; CNN and BiLSTM share that vocabulary within the setting. The response-only vocabulary contains 15,053 entries. All word embeddings are learned from scratch.

Prompt-only inputs allow 128 tokens and response-only inputs allow 384. Combined inputs reserve both budgets and insert a separator, for a maximum of 513 tokens. In the completed data, no prompt or response exceeds its respective limit, so no examples are truncated. Padding is handled explicitly in both architectures.

Architectures

The CNN embeds tokens into 128 dimensions, then applies one-dimensional convolutions with widths 3, 4, and 5 and 128 filters per width. ReLU activations are followed by max pooling over valid token windows. The three pooled vectors are concatenated into 384 features, followed by dropout and a linear classifier with three output logits. Masking prevents padded windows from determining the maximum.

The BiLSTM uses 128-dimensional embeddings and one bidirectional recurrent layer with 128 hidden units per direction. Packed sequences prevent padding from affecting recurrent states. The final forward and backward hidden states are concatenated into 256 features, followed by dropout and a three-class linear classifier.

Training setting	Value
Loss and optimizer	Cross-entropy and AdamW
Learning rate and weight decay	0.001 and 0.0001
Batch size and dropout	64 and 0.3
Gradient clipping	Maximum norm 1.0
Epoch limit and patience	10 epochs and 3 epochs
Seed and execution	42; CPU; four threads; PyTorch 2.14.1

Table 2. The same training settings were used for both architectures and all feature settings.

Selection and evaluation

After each epoch, validation macro-F1 is calculated. The checkpoint with the highest value is saved, with lower validation loss breaking ties. Training stops after three consecutive epochs without improvement under this rule or at epoch ten. Test evaluation uses the selected checkpoint. Accuracy measures the fraction of correct labels. Macro-F1 averages the F1 scores of the three classes equally; each class F1 balances precision and recall. Only seed 42 has been evaluated, so no across-run standard deviation is reported.

4. Classification results

Model	Features	Accuracy	Macro F1	Best / run epochs
CNN	Prompt only	33.33%	0.1667	7 / 10
BiLSTM	Prompt only	33.33%	0.1667	1 / 4
CNN	Response only	98.30%	0.9830	6 / 9
BiLSTM	Response only	95.78%	0.9575	10 / 10
CNN	Prompt + response	98.52%	0.9852	10 / 10
BiLSTM	Prompt + response	94.15%	0.9411	5 / 8

Table 3. Test results on 1,350 examples. Best / run epochs gives the selected epoch and total epochs completed. All runs use seed 42.

RQ1 Attribution from responses

Both architectures identify the generating model well from the response alone. CNN correctly labels 1,327 of 1,350 responses, compared with 1,293 for BiLSTM. This is a 2.52 percentage-point accuracy advantage for CNN in this run. Response-only validation macro-F1 is 0.9822 for CNN and 0.9599 for BiLSTM, close to the corresponding test scores of 0.9830 and 0.9575. This agreement supports the held-out result within the current data construction.

RQ2 Contribution of the prompt

Prompt-only accuracy is exactly 33.33% because every prompt occurs equally often under each label. Both trained classifiers predict GPT for every test example. That gives GPT an F1 score of 0.5 and the other two classes an F1 of zero, producing macro-F1 of 0.1667. The prompt-only control therefore behaves as expected.

Adding the prompt changes CNN accuracy from 98.30% to 98.52%, only three additional correct predictions. BiLSTM accuracy decreases from 95.78% to 94.15%, or 22 fewer correct predictions. These outcomes provide no consistent evidence that adding the prompt improves attribution. The small CNN gain should be interpreted as a result of this run, without a claim of statistical reliability.

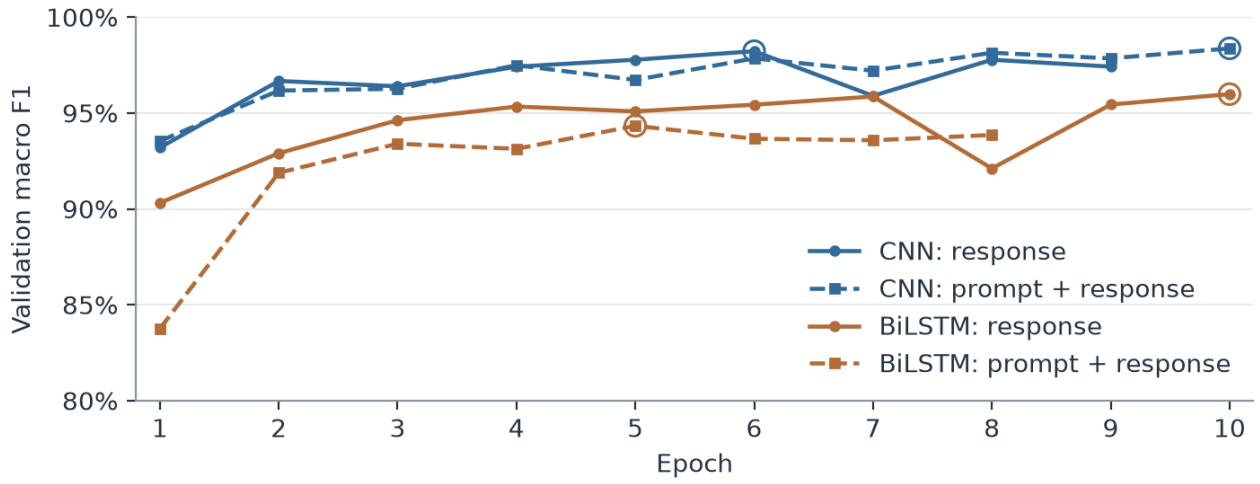


Figure 1. Validation trajectories for the four experiments containing responses. Outlined circles mark selected checkpoints. The last epoch need not be the selected epoch.

5. Error analysis

The confusion matrices show that performance is strong across all three classes rather than being driven by one majority label. Each true class has 450 test examples. For the response-only CNN, class F1 scores are 0.9767 for Claude, 0.9922 for GPT, and 0.9799 for Gemini. GPT is the easiest class for both response-only models under this setup.

The response-only CNN makes 23 errors: nine Claude responses, two GPT responses, and twelve Gemini responses. Its most common confusion is Gemini predicted as Claude, with eleven cases. The response-only BiLSTM makes 57 errors, including 39 Claude responses. Its Claude recall is 91.33%, compared with 98.00% for CNN.

Adding the prompt does not improve every class. CNN total errors fall from 23 to 20, but Claude errors rise from nine to twelve. BiLSTM total errors rise from 57 to 79, with 34 Claude responses predicted as Gemini in the combined setting. These counts describe the observed failure patterns; they do not establish which individual words or structures caused a mistake.

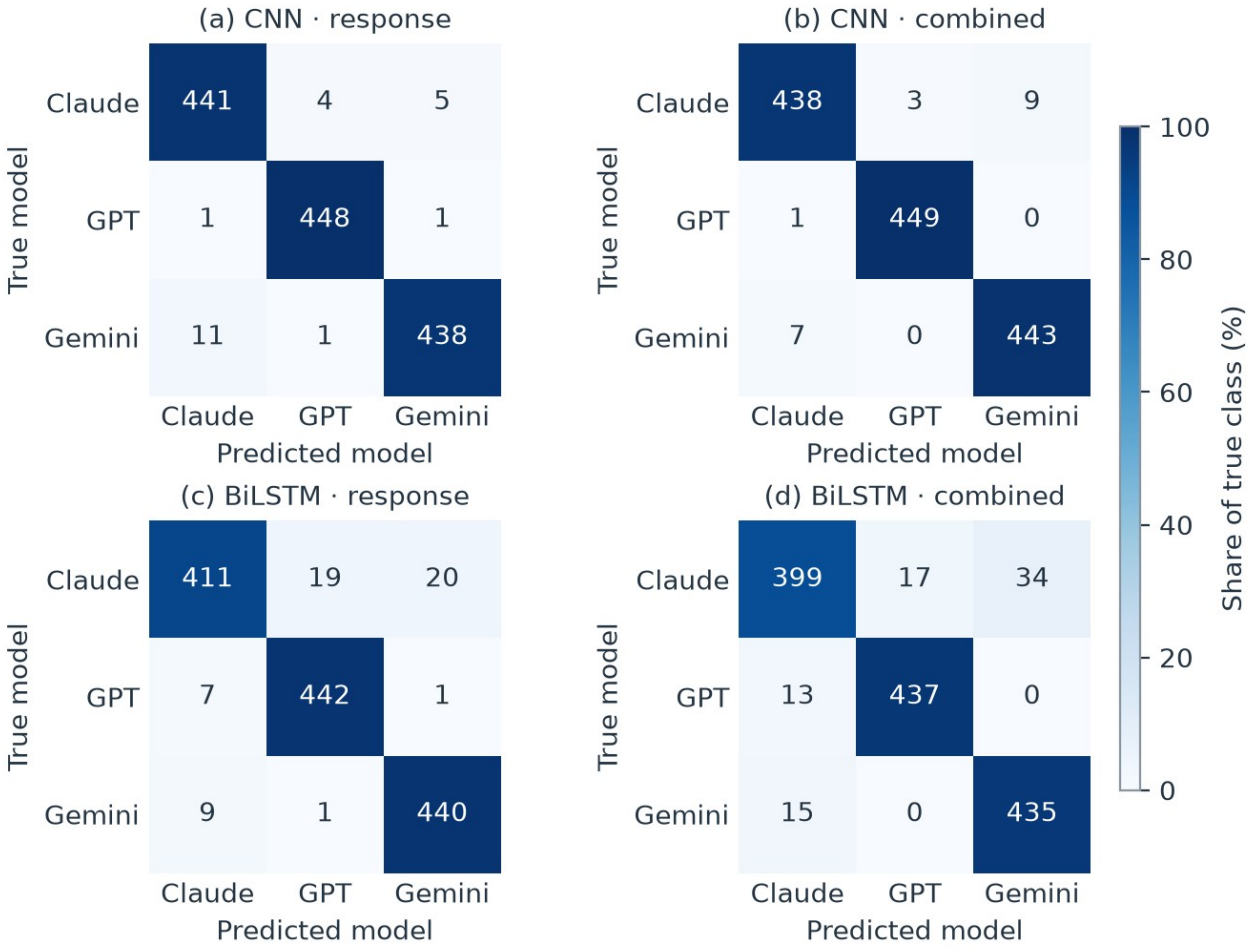


Figure 2. Test confusion matrices. Combined means prompt plus response. Cells show counts; color shows the percentage within each actual class. Each row contains 450 test examples.

6. RQ3 Transfer between scientific domains

The transfer experiment uses an explicit keyword mapping because the supplied category field contains mismatches, such as mammals labeled Astronomy and urology labeled Finance. Before the new fits, 76 keywords were assigned to life sciences and 83 to physical sciences. The remaining 141 were excluded from this extension because they were outside these domains or crossed the chosen boundary. The mapping is saved with the code; original responses and train, validation, and test membership were retained.

Domain	Train rows / topics	Validation rows / topics	Test rows / topics
Life sciences	1,590 / 53	270 / 9	420 / 14
Physical sciences	1,800 / 60	420 / 14	270 / 9

Table 4. Each topic contributes 30 responses, ten per generator. The source test is the other direction's target test.

CNN and BiLSTM were each retrained in both directions using the original response-only architecture, optimizer, seed, and epoch limits. Only the source training split supplied examples and vocabulary. Only source validation macro-F1 selected the checkpoint. That same checkpoint was evaluated on both its own held-out domain and the other domain, without target adaptation. Normalized keywords, prompts, and exact response texts are disjoint across the four data partitions.

Train domain	Model	Same domain Acc / F1	Other domain Acc / F1	Acc gap pp 95% interval
Life sciences	CNN	96.19% 0.9617	94.44% 0.9445	-1.75 [-4.84, +1.24]
Life sciences	BiLSTM	91.67% 0.9151	89.26% 0.8906	-2.41 [-6.67, +1.93]
Physical sciences	CNN	95.93% 0.9590	95.24% 0.9521	-0.69 [-3.25, +1.96]
Physical sciences	BiLSTM	95.56% 0.9553	89.29% 0.8911	-6.27 [-11.22, -1.08]

Table 5. Transfer accuracy and macro-F1 for the selected source checkpoints. Gap is other-domain minus same-domain accuracy in percentage points; brackets give 95% keyword-bootstrap intervals.

CNN reaches 94.44% accuracy for life-to-physical transfer and 95.24% in the reverse direction. BiLSTM reaches 89.26% and 89.29%. All four are well above the balanced three-class chance level of 33.33%. The largest observed accuracy decrease is physical-to-life BiLSTM: 6.27 percentage points, with a conditional 95% interval for the gap of [-11.22, -1.08]. The other three transfer-gap intervals include zero; this does not establish equivalence between domains.

Claude has the lowest class F1 in all four cross-domain evaluations, ranging from 0.8333 to 0.9348. GPT has the highest class F1 for the transferred CNN models, whereas Gemini is strongest for the transferred BiLSTM models. The evidence supports subject-transferable attribution signals under this shared generation protocol, with differences across architectures and generators.

The two directions use different training sizes and different test keywords. A transfer gap therefore reflects this entire evaluation setup; it cannot isolate an intrinsic difference in domain difficulty. The mapping and protocols were fixed before the new results, but the original test scores had already been inspected. RQ3 and RQ4 are exploratory extensions rather than fresh confirmatory holdouts.

7. RQ4 Response characteristics

To investigate possible attribution signals, response characteristics were measured on the training split only, using 2,100 responses per model. Token length uses the same word-and-punctuation tokenizer as training. The colon and semicolon measures are the percentage of responses containing at least one such character. These are descriptive statistics, not classifier feature-importance scores.

Generator	Mean tokens	Median tokens	Contains colon	Contains semicolon
Claude Sonnet 4.6	109.3	110	22.4%	35.5%
GPT-5.4	118.4	118	56.5%	45.7%
Gemini 3.1 Pro	178.8	183	0.2%	0.2%

Table 6. Training response statistics, with 2,100 examples for each generator. Punctuation percentages use raw response text.

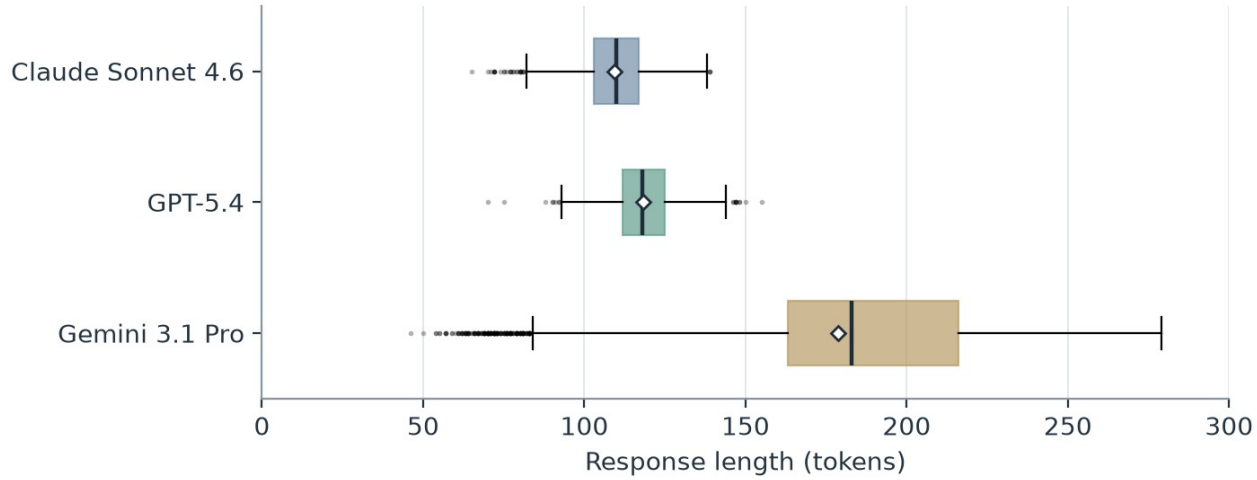


Figure 3. Training response lengths, including punctuation tokens. Boxes span the first to third quartiles; lines mark medians and diamonds mark means. Whiskers reach observations within 1.5 interquartile ranges of the box; dots show outliers.

Gemini responses are substantially longer on average than those of Claude or GPT. Gemini uses colons and semicolons in only about 0.2% of responses each, whereas both characters are much more common in the other two groups. GPT has a particularly high rate of colon use. These differences motivate the controlled comparisons below.

Interpretable feature classifiers

Three multinomial logistic classifiers were fitted to the original 6,300 training responses. The length baseline uses log(1 + word count) and log(1 + character count). The structure baseline uses 18 structural descriptors, including punctuation, capitalization, mean word length, and unique-word fraction, with no raw length-count feature. These descriptors can still correlate with response length. The lexical baseline uses word unigram and bigram TF-IDF. Scaling, vocabulary, and IDF are fitted on training data only. C is selected from 0.01, 0.1, 1, and 10 using validation macro-F1, with validation log-loss breaking ties; no validation examples are added during fitting.

Feature set	Accuracy	Macro F1	Selected C
Length only	73.26%	0.7394	0.01
Structural descriptors	89.70%	0.8967	1.0
Word and phrase TF-IDF	98.89%	0.9889	10.0

Table 7. Feature baselines evaluated on the original 1,350 test responses. All model selection uses the original validation split.

Length alone reaches 73.26% accuracy, showing that response size carries substantial label information. Structural descriptors reach 89.70%, and word/phrase TF-IDF reaches 98.89%. The lexical classifier is a strong additional baseline in this run; its score is not evidence of a reliable advantage across training seeds. These feature groups overlap statistically, so their accuracies cannot be added to assign independent contributions.

Positive lexical coefficients include “across” and “via” for Claude, “test” and “feasible” for GPT, and “will” and “will be” for Gemini. These examples suggest differences in recurring wording and phrasing. The structure classifier also includes unique-word fraction as a simple lexical-diversity measure, alongside punctuation and word-shape features. Full signed coefficients are saved for every generator.

The structure classifier includes raw capitalization and newlines, which the original lowercase neural tokenizer ignores. Its success therefore does not show that the neural models use those particular cues. Feature coefficients show associations within this corpus, not universal author signatures or causal feature importance.

Removing punctuation and controlling token count

Both neural architectures were retrained under two prespecified transformations, keeping all original rows, labels, splits, hyperparameters, and seed. The first removes punctuation and retains the complete lowercase word sequence. The second also restricts every response to its last 32 word tokens. Every example is at least 32 words, so no rows are filtered or padded to meet this budget. A first-32-word diagnostic found one identical train/validation prefix; the suffix rule was chosen before new model results. Both final transformations have zero cross-split exact duplicates.

Response text	Model	Accuracy	Macro F1	Acc gap pp 95% interval
Original response	CNN	98.30%	0.9830	Baseline
Original response	BiLSTM	95.78%	0.9575	Baseline
No punctuation	CNN	96.30%	0.9629	-2.00 [-2.81, -1.19]
No punctuation	BiLSTM	94.59%	0.9458	-1.19 [-2.59, +0.22]
No punctuation Final 32 words	CNN	92.44%	0.9247	-5.85 [-7.56, -4.37]
No punctuation Final 32 words	BiLSTM	92.44%	0.9248	-3.33 [-5.19, -1.41]

Table 8. Retrained neural ablations on the same 1,350 test rows. Gaps compare each transformation with its architecture’s original response-only result. Intervals use paired keyword resampling.

Removing punctuation reduces observed accuracy by 2.00 percentage points for CNN and 1.19 for BiLSTM. The paired interval for the BiLSTM difference includes zero, so that smaller decrease is uncertain under keyword resampling. Restricting the punctuation-free input to 32-word suffixes gives 92.44% for both architectures, decreases of 5.85 and 3.33 points from their original results. Word sequences remain highly predictive even after punctuation and token-count variation are removed.

Fixed 32-word excerpts remove variable token counts, but also discard earlier content and retain only the end of each response. The score change cannot be attributed solely to length. Likewise, retraining on punctuation-free text measures the information remaining after transformation, not the causal dependence of the original trained network on punctuation. Word choice, word length, and word order remain available in both ablations.

8. Limitations and interpretation

The original test set contains unseen keywords, and RQ3 additionally separates life-science and physical-science subjects. All examples still use the same scientific-ideation task, standard prompt family, low effort setting, and generator versions. The study does not establish transfer to coding, dialogue, or factual questions. Domain membership describes the requested keyword; an individual generated idea can still combine fields.

Only three specific model versions are represented. The classifiers are closed-set predictors that must choose one of these labels. There are no human-written examples or outputs from unknown generators, so these results cannot be interpreted as general AI detection. A later model update could also change the learned signals.

Matching the low effort label across providers does not guarantee equivalent reasoning budgets or sampling behavior. The dataset was generated under a particular structured-output protocol. Length and formatting differences may reflect that protocol as well as the underlying models. Attribution accuracy here should be understood as identification under these generation conditions.

The prompt feature is reconstructed from a pinned published template and the recorded keyword. It is not independently recovered from a separately stored prompt string for every response. Exact-text and keyword separation reduce direct overlap, but related concepts and semantic paraphrases may remain across splits. Ten responses from a shared keyword also form a cluster; the 1,350 test rows are not 1,350 independent topics.

All neural experiments use seed 42 and one prepared split. The extension reports 95% percentile confidence intervals from 2,000 bootstrap resamples of whole keyword groups, retaining all 30 responses per sampled keyword. RQ4 differences use paired resamples of the same 45 test keywords. RQ3 resamples its two test domains independently when estimating transfer gaps. These intervals condition on the fitted model and omit training-seed, split-selection, and domain-mapping uncertainty. Only 9 physical-science and 14 life-science test keywords are available, so the transfer intervals have limited resolution. No repeated-seed standard deviation or hypothesis test is reported.

9. Lessons and implementation experience

My main takeaway is that tuning the code means making the comparison meaningful, not just increasing accuracy. The prompt-only experiment helped clarify this: its 33.33% accuracy is expected because identical prompts have all three labels. Macro-F1 of 0.1667 also shows why accuracy alone can hide predictions concentrated on one class. The data split, evaluation metrics, and baseline comparisons are therefore important parts of the implementation.

The training histories also show why more epochs do not always give a better model. The response-only CNN selected epoch six, although training continued to epoch nine. A useful tuning procedure is to compare settings on validation data and save the strongest checkpoint before evaluating the test set. I would use that procedure to compare learning rates or dropout in future work; this project kept neural hyperparameters fixed.

The work also illustrates the value of a reproducible data pipeline. Reading only the three idea files avoids downloading unrelated embeddings and judge results. Pinning source revisions, recording hashes, and separating keywords make the choices traceable. The original six runs took 28.46 minutes of recorded training and evaluation time on CPU. The RQ3 and RQ4 extensions add eight neural fits and three logistic classifiers; the neural fit times are recorded separately in the saved results.

10. Conclusion

Response-only classification answers RQ1 positively within this dataset: CNN reaches 98.30% accuracy and BiLSTM 95.78%. RQ2 finds no consistent improvement from adding prompts. RQ3 shows substantial transfer between the selected scientific subject groups, especially for CNN, but does not test transfer to different tasks. RQ4 finds predictive information in length, structural descriptors, and word choice. Both neural classifiers retain 92.44% accuracy on punctuation-free, equal-token-count suffixes. The ablations reduce observed performance while preserving strong attribution: punctuation and variable token count are not the only available attribution signals. Repeated training seeds, new keyword splits, and a different task would test how stable these results are.

Appendix

The project includes prepare_data.py, train.py, analyze_results.py, prepare_rq3.py, rq3_experiments.py, rq4_experiments.py, and analyze_extra_credit.py. The original splits remain in data/prepared; audited domain lists are in assets/rq3_domains.json and derived splits in data/rq3. Original results remain in runs/results.csv. Transfer and ablation results are saved separately in runs/rq3 and runs/rq4. The extension analysis saves prediction audits, confidence intervals, contrasts, and comparison figures in runs/extra_credit_analysis.

All 25 implementation tests passed before packaging. They covered split balance and duplicate grouping, training-only vocabulary and feature fitting, token budgets, padding and gradient updates, domain membership, text transformations, prediction integrity, bootstrap calculations, and analysis paths. An independent audit replayed all 21 saved test evaluations from 14 neural and three logistic fits and matched all eight report tables. The extension analysis checked held-out IDs, keywords, labels, probabilities, and saved metrics for its 17 response-based evaluations. These checks validate the pipeline; they do not measure training-seed uncertainty.

Use Python 3.11 or 3.12 and run these commands from the llm_authorship directory. They use the supplied prepared splits. The runs_new and data/rq3_new directories must not already contain experiments.

```
python -m pip install -r requirements.txt
python train.py --output-dir runs_new --device cpu
python analyze_results.py --runs-dir runs_new \
    --output-dir runs_new/report_figures
python prepare_rq3.py --output-dir data/rq3_new
python rq3_experiments.py --data-dir data/rq3_new \
    --output-dir runs_new/rq3
python rq4_experiments.py --baseline-dir runs_new \
    --output-dir runs_new/rq4
python analyze_extra_credit.py --baseline-dir runs_new \
    --rq3-data-dir data/rq3_new --rq3-dir runs_new/rq3 \
    --rq4-dir runs_new/rq4 --output-dir runs_new/analysis
```

These commands reuse the supplied splits and write to fresh directories. The ZIP contains prepared data, code, predictions, metrics, and analysis; raw downloads, environments, and fitted weights are omitted. Retraining recreates the checkpoints. Always use new experiment directories when reproducing runs from the ZIP. Seeds 43 and 44 were not run. Data is distributed under CC BY 4.0; the upstream prompt uses the MIT license. The notice in assets/UPSTREAM_LICENSE.txt is copied verbatim from the upstream source, including its author placeholder.

References

[1] Yu Chinen and Haruka Ozaki. 2026. On the Effects of Reasoning Effort and Prompt-Based Diversification on Scientific Ideation Diversity. Discovery Science 2026. Associated dataset: Scientific Ideation Diversity. DOI: 10.57967/hf/10818. https://huggingface.co/datasets/tax-free/scientific-ideation-diversity

[2] bioinfo-tsukuba. Scientific Ideation Diversity source code and generation prompts. https://github.com/bioinfo-tsukuba/scientific-ideation-diversity.